From arithmetic to groups and algebras

If you are not used to Mathematica: evaluate one input cell at a time, from top to bottom, with Shift+Enter.
You can read the outputs below first, and then try evaluating each cell yourself (Shift+Enter).

Arithmetic is the familiar system of rules for combining numbers by operations such as addition and multiplication that you learned in school, from Greek "*arithmetike tekhne*", meaning "*the art of counting*".

1. Two familiar operations

Addition combines numbers: 2 + 3 = 5.

In[4071]:=
```
2 + 3
```
Out[4071]=
5

In Mathematica, we usually write multiplication with a space, which is shown as a grayed-out cross "×":

In[4072]:=
```
2 × 3
```
Out[4072]=
6

Multiplication can also be written "*", it's the same as the above (try exchanging the symbol).

An *identity operation*, like an identity matrix, is the operation that does nothing.
For addition, identity is addition by 0. For multiplication, identity is multiplying by 1. Note that these two identity operations are different in detail.

In[4073]:=
```
{0 + 7, 1 × 7}
```
Out[4073]=
{7, 7}

Here { } denotes a "list": as it sounds, a list of objects with no additional structure. For now, think of a list as a "computer science thing", not a physics thing in itself. (Later, we will actually ascribe some more meaning to lists.)

If you're not used to Mathematica, try replacing "7" above with another integer, and run that cell again (Shift+Enter) to get started experimenting.

An *inverse operation*, like an inverse matrix, reverses an operation, i.e. performs it backwards. An additive inverse of 3 is –3 because their sum is 0:

In[4074]:=
```
3 - 3
```
Out[4074]=
0

A multiplicative inverse of 3 is 1/3 because their product is 1:

In[4104]:=
```
3 / 3
```
Out[4104]=
1

Note that again, the details of the two inverse operations are quite different.

In Mathematica (as in many programming languages), the symbol == is a *test* whether two expressions are equal, whereas a single = is *assignment*, i.e. assigns a name to a value.
The next cells use the "equal test" == to compare two ways to put parentheses:

In[4076]:=
```
associativityTest = {(2 + 3) + 4 == 2 + (3 + 4), (2 × 3) 4 == 2 (3 × 4)}
```
Out[4076]=
{True, True}

and two ways to order the objects:

In[4077]:=
```
commutativityTest = {2 + 3 == 3 + 2, 2 × 3 == 3 × 2}
```
Out[4077]=
{True, True}

The first pair tests **associativity**: moving parentheses. The second tests **commutativity**: swapping order.

Both properties are true for ordinary numbers. But they don't always hold, i.e. for other objects than ordinary numbers. You should be familiar with the non-commutativity of matrices. Non-associativity is probably less familiar.
Also keep in mind that examples illustrate a law, but of course don't prove it for every number. We will typically not prove statements like the above in general, but invite you to try examples to get a feeling. Proofs would be discussed in courses in "abstract algebra" (at KaU, the course *Algebraiska strukturer*), which typical physics students never take.

2a. The chosen set matters

A set is a "collection of allowed values". An operation is *closed* on a set when applying it to two allowed values always gives another allowed value. To call a set with an operation a **group**, we also need an **identity**, an inverse inside the set for every member, and associativity. (Note that commutativity is an *additional* requirement, not part of the group requirement.)

For addition, the integers $\mathbb{Z}$ form a group: sums of integers stay integer, 0 is the identity, and every integer has a negative. For multiplication, the integers have no inverse: for example, the inverse of 2 is 1/2, not an integer.
On the other hand, the nonzero rational numbers $\mathbb{Q}^\times$ do form a group under multiplication. We exclude 0 because no number multiplied by 0 gives 1.

In[4078]:=
```
{2 - 2, 2/2}
```
Out[4078]=
{0, 1}

Both calculations reach the identity appropriate to their operation. But again, 1/2 is outside the integers, so it cannot serve as a multiplicative inverse there. Multiplication by 0 always gives 0. Changing the number we multiply by cannot make 1.

In[4079]:=
```
{0 × 2, 0/2, 0 × 100}
```
Out[4079]=
$\{0, 0, 0\}$

2b. Two very small examples

Example: choose just the set {–1, 1} and multiplication. Its four possible products are shown in the order (-1)·(-1), (-1)·1, 1·(-1), 1 · 1. Every answer is still in the set. The identity is 1, and each member is its own inverse.

In[4080]:=
```
{-(-1), -1, 1 (-1), 1 × 1}
```
Out[4080]=
$\{1, -1, -1, 1\}$

Example: {0,1} with addition is not closed: 1+1 leaves that set.

In[4081]:=
```
{0 + 1, 1 + 1}
```
Out[4081]=
$\{1, 2\}$

3. Preparation for lecture 3: small matrices

Although we shouldn't look forward *too* much, the following can help to understand the point of the discussion of arithmetic above, that might have seemed very boring. You are already familiar with the generalization of ordinary numbers to matrices. But maybe you haven't thought very much about the choices that were made in this generalization. Thinking about those choices might help us see what **other kinds of generalizations could be interesting**, than specifically going from numbers to matrices (that you already know).

Consider 2×2 matrices. Here *"id"* is the identity matrix (capital I in Mathematica is reserved for the imaginary unit), *x* swaps the two coordinate positions (as the Pauli matrix σ_x), and *z* changes the sign of the second (as the Pauli matrix σ_z). A matrix is stored as a list of lists . (The semicolon ";" below separates assignments and hides their immediate output.)

In[4082]:=
```
id = {{1, 0}, {0, 1}}
x = {{0, 1}, {1, 0}}
z = {{1, 0}, {0, -1}}
```
Out[4082]=
$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

Out[4083]=
$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$

Out[4084]=
$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

Note the assignment of the grouping of the list of lists as rows vs. columns.
As you know, *matrix addition* adds the corresponding entries of each matrix:

In[4085]:=
```
id + x
```
Out[4085]=
$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

...and *multiplication by an ordinary number*, like 2, scales every entry (the number is a "scalar"):

In[4086]:=
```
2 x
```
Out[4086]=
$$\begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}$$

Reflect for a moment that these two operations, *matrix addition* and *scalar multiplication*, are the obvious generalizations of the elementary operations *addition* and *multiplication* on ordinary numbers, now used on a different (here "bigger") object.
But how to generalize **multiplication by matrices** (as opposed to scalar multiplication) is less obvious.

Of course, you know the answer. A lower dot between matrices, as `x.z`, means matrix multiplication in the usual sense:

In[4087]:=
```
x.z
```
Out[4087]=
$$\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

But now, contrast ordinary multiplication ("`*`" in Mathematica, or just the "space" key), which is *not* matrix multiplication, but the product of two lists:

In[4094]:=
```
x * z
```
Out[4094]=
$$\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

The product of two lists, unlike matrix multiplication, works entry by entry:

In[4101]:=
```
{a, b} * {c, d}
```
Out[4101]=
$$\{a\,c,\, b\,d\}$$

If you think Mathematica is crazy for making this choice for the meaning of "`*`", because you think matrix multiplication is more important than multiplication of lists, you would be missing the point: it is useful to have *both* of these operations available. (In Python, there are much crazier, and also useful, choices like this!) In particular: **conventional matrix multiplication is a choice**.

As you know, matrix multiplication is not commutative, in general. The order of x and z matters, i.e. they don't commute. As above, the test "==" reports whether each claim holds.

In[4097]:=
```
x.z == z.x
```
Out[4097]=
False

(With the other meaning of multiplication of lists above, clearly it is commutative)

In[4102]:=
```
x * z == z * x
```
Out[4102]=
True

Distributivity connects addition and multiplication, here matrix multiplication. The next equality says that multiplying a sum by *z* on the right gives the same result as multiplying each summand by Z and adding.

In[4090]:=
```
(id + x).z == z + x.z
```
Out[4090]=
True

Groups and algebras: transformations and linear combinations

Inspired by matrices, we can now see at least two different generalizations of ordinary arithmetic, that have names.

A ***group*** essentially uses the idea of multiplication but leaves out addition. Operations can be combined (multiplied), there is an identity, and every operation can be undone (has an inverse). An example is rotations. If we first rotate something and then rotate it again, the two rotations compose to give another rotation. Every rotation can also be undone. But "linear combination" of rotations is not defined. It would be like performing several *different* rotations *simultaneously*, as opposed to in sequence. (See below for more comments on this.)

An ***algebra*** contains more structure. Its elements can be added and multiplied by ordinary numbers, as in a vector space. They can also be multiplied with each other. Matrices are the main example for us.

More specifically, **all 2×2 complex matrices together form an a (noncommutative) *algebra***: we can add matrices, multiply them by complex numbers, and multiply matrices among each other.

However, **all matrices do *not* form a *group* under multiplication**. Obviously, the zero matrix has no inverse. But the key new thing is that there are nonzero matrices that also have no inverse, as operations: for example, id – *x* is a nonzero matrix without an inverse.

In[4091]:=
```
(id - x)
```
Out[4091]=
$$\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$$

Multiplying id – *x* and the nonzero matrix id + *x* gives the zero matrix:

In[4092]:=
```
(id - x).(id + x)
```
Out[4092]=
$$\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

Since there exists at least one matrix (here id–*x*) that sends a nonzero matrix to the zero matrix, we can "get stuck", i.e. we cannot recover the original matrix we multiplied (here id+x). In other words, we "lost information" of what the original matrix we multiplied was, since the transformation cannot be inverted. Information loss appears in many places in physics (e.g. entropy), so don't view it as something "wrong" in itself. Just note this as a fact, something possibly interesting.

Actually, you already know how to proceed. *How can we restrict 2x2 matrices to make them invertible?* Require nonzero determinant. The "det ≠ 0" matrices do form a group, that we will call "GL(2)", for "General Linear" (2 x 2 transformation). This restriction excludes the matrix example above:

In[4103]:=
```
Det[id - x]
```
Out[4103]=
$$0$$

Summary and outlook: group algebra

The extra "linear structure" of addition (linear combination) that algebras have, but groups do not, is often very useful in physics. Quantum states can be superposed, differential equations used in physics are often linear, and operators can be added and multiplied. If *A* and *B* are matrices, then 3*A* – 2*B* is another matrix. There is no analogous operation in an abstract group: if *g* and *h* are two group elements, then as noted above, a "linear combination of operations", such as 3*g* – 2*h,* has no meaning. As mentioned above, for rotations, a "linear combination" would be something like performing several *different* rotations *simultaneously*, as opposed to in sequence. "Simultaneous rotation" (or more precisely, superposition of different possible rotations) does not seem to make sense.

As you can maybe guess from this formulation, we will in fact assign meaning to superpositions of operations. This is called a *group algebra,* to be defined. Here the point is not to define this new thing, but to point out that such structure is *not* included in ordinary groups or algebras, in the sense that neither groups nor algebras separately describe everything that a group algebra is. Instead, understanding ordinary groups and algebras is a necessary starting point.